

Detecting is easy. Closing the loop is the problem

Digital twin

Remediation

17 September 2026 · 9 minute read

Almost the entire network tooling industry stops at diagnosis: it tells you what is wrong and the product ends there. This article is about what comes next, why \u201cjust let it fix itself\u201d is a bad idea, and what it takes for a finding to actually close rather than merely be documented.

Finding the problem is the easy part

Almost the entire network tooling industry stops at diagnosis. They tell you the device at the north site is missing its secondary time server, that the management access list still permits a range it should not, that the operating system version has fallen behind. That is useful, and it is where the product ends.

What comes next is an engineer opening a session to each device, typing the commands by hand, hoping not to make a mistake, and noting somewhere that it is done. On a network of twenty devices it is tedious. On one of two hundred, the finding list grows faster than it gets closed, and the compliance report becomes a document nobody reads because it always says the same thing.

A finding that cannot be closed from the same tool that found it is a to-do item, not a finding.

Why "just let it fix itself" is not the answer

The natural reaction is to ask the tool to correct things automatically. And that is exactly where care is needed, because a badly executed automatic change on the network does not raise a ticket: it raises a site visit.

There are three concrete risks, and they are different from each other:

- **The command may be invalid.** Correct syntax depends on the platform, the version, and sometimes the model. A command that works on one version is rejected on another, and if the change was half applied the device is left in an intermediate state nobody declared.
- **The command may be valid and wrong.** Applying a management access list that omits the address you connect from is syntactically perfect and locks you out of the device.
- **The change may be correct and badly timed.** Fixing a spanning tree priority is the right thing to do, and doing it at eleven on a working morning triggers a reconvergence nobody asked for.

That is why closing the loop does not mean automatic. It means every step — propose, review, approve, apply, and verify — happens inside the same tool, with the person placed where their judgement actually contributes.

The digital twin: compare before you touch

The term gets used for many things. Here it means something concrete and verifiable: a representation of how a device *should* be configured according to declared intent, which can be compared against how it is actually configured.

That comparison produces a diff: the lines to add, the lines to remove, and the ones that change. And that diff is the artefact reviewed **before** anything touches the network. It is not a promise of what the tool will do; it is the exact text it is going to send.

The difference from approving a change described in prose is enormous. "Fix the device's time configuration" is an intention. The three concrete lines that will be sent, in the vendor's own syntax, are something an engineer can read in fifteen seconds and approve or reject on solid grounds.

This is also where most multi-vendor tools fall down, for a reason we covered in [the article on source of truth](#): if the internal model is a translation into a common syntax, the diff you are shown is not the one that will be applied. It resembles it. And "resembles" is not a basis for approving a change in production.

Every vendor has its own safe way to apply

Applying a change is not opening a session and pasting text. Each platform offers a primitive designed precisely so that a remote change does not lock you out, and using it is the difference between a controlled operation and a gamble.

- **Junos** has deferred confirmation: the change is applied and rolls back on its own unless someone confirms it within a time limit. If the change locked you out, the device returns to its previous state by itself.
- **IOS and IOS-XE** have configuration archive and replace, which allows a return to a known state without rebuilding it by hand.
- **FortiGate** is driven through an API, where the device's response confirms whether the object ended up as requested, instead of having to interpret terminal output.

A tool that sends text over a terminal to all three alike wastes all three protections. Respecting each vendor's primitive costs more development effort, and it is the only way a remote change is genuinely reversible.

The step that is almost always missing: verify

Applying is not closing. A device can accept the command and still not end up as expected: a dependency that did not exist, a referenced object that was not declared, a configuration that is accepted but does not take effect until a process restarts.

The loop only closes when the same device is audited again against the same rule and the finding is gone. That turns "the change was applied" — a claim about what the tool did — into "the finding is closed", which is a claim about the state of the network. They are different things, and only the second belongs in a report.

Detect, propose, approve, apply, verify. Without the last step, the other four are a documented assumption.

How Oktavia handles it

Oktavia's Digital Twin generates a device's intended configuration from intent declared by scope and compares it against the running one. The diff is reviewed on screen, approved, and only then applied using the primitive that belongs to that vendor. Afterwards, a fresh audit against the same rules confirms whether the finding closed.

The whole path leaves downloadable artefacts — the proposed configuration, the diff, the result of the application, and the follow-up audit — so evidence for an internal or regulatory audit is not assembled afterwards: it was produced while the work happened.

The same applies to provisioning new services, not only to remediation: a tunnel between two sites, advertising a network into the routing protocols, dynamic address assignment. Each one with its rollback procedure defined before it is applied.

How many open findings do you have today?

If the answer is "I don't know" or "the same as last quarter", detection is not your problem. To quote, all we need is roughly how many devices you manage.

About Oktanet

Oktanet is a Mexican network automation company focused on controlled, traceable, and scalable operations for enterprise environments across Latin America.

[LinkedIn](#) · [YouTube](#)

© 2026 Oktanet · Oktavia · Public article, with no confidential customer information.